(Chapter 10)

Batch Processing

Services → (Online System) → request comes, server responds with a response. (Request-Response) model

Batch Processing System (offline system) →

Taking a large amount of data and usually there is no user waiting for a response at that moment. Using the input data a group is created and batched and output is given.

Stream Processing (Near Realtime) → Stream of events come and need to be processed like click stream and maybe some dashboard has to be displayed.

Now we take an example of Batch Processing with Tools.

Lets say we have a access log of a access of a website

timestamp GET domain —— url

7th (field)

In two ways we are processing this log file.

Unix tools	Java Code
1) cat access-log | ① → access the log file awk '{print $7}' | ② ⇒ extract 7th element of each entry sort | ③ ⇒ sort by url lexicographically uniq -c | ④ ⇒ count by urls sort -r -n | ⑤ ⇒ sort by their frequency head -n 5 ⑥ ⇒ Extract only top 5 2) UNIX is pretty fast to process this data like GBs of files in second.	1) Map<String, Integer> mp = new HashMap ↗ go through the file and enter it 2) You can only process these if the file is smaller than in-memory RAM.

Comparing Java Code V/S UNIX script

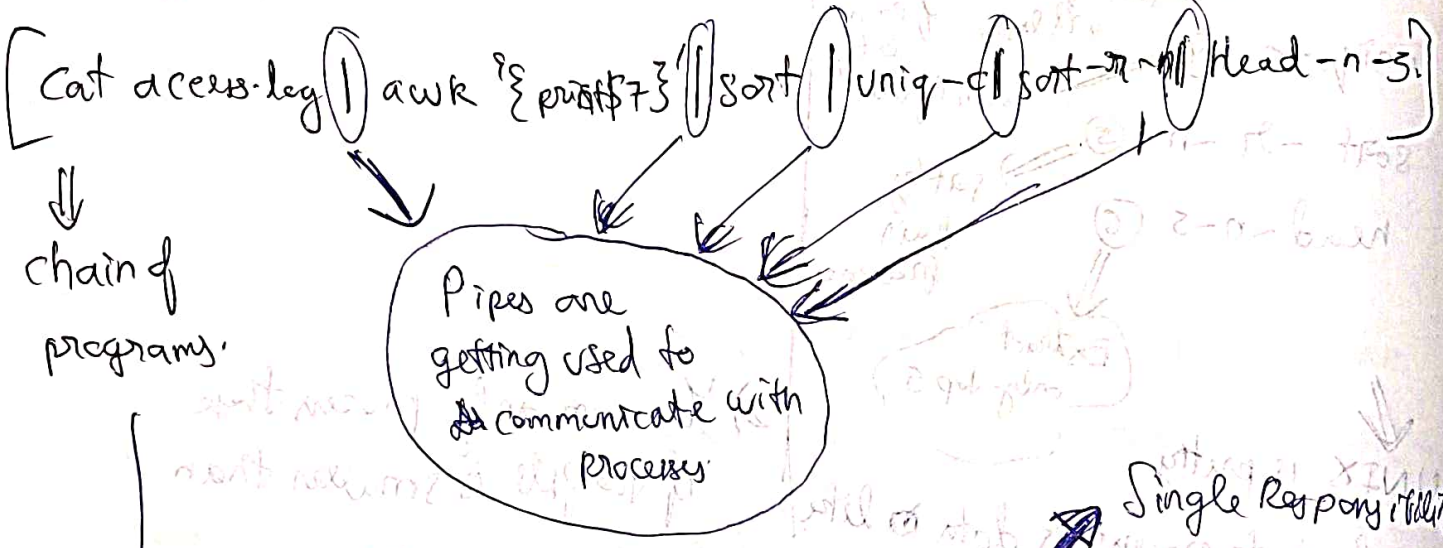
We saw in Java we have a memory limit.

Whereas is script of UNIX no limit, also sorting happens in Disk like mergeSort → {As in SSTables in Cassandra}. And since it merging two ~~sorted~~ contiguous segments of a array or file, so access pattern is sequential. Disk seeker does not have to move much and movement is minimum. Sorting gets parallelized across multiple CPU cores. Disk sorting

UNIX philosophy

The inventor of UNIX said that, make processes or programs talk to each other using pipes.

Hence pipes started getting used for communication.



* Make each program do 1 thing well, to do something well write another program don't change current one → Single Responsibility
* Expect output of 1 program to be input of another program, → format same for communication
  Don't complicate by using interactive input
* Redesign if program is clumsy, keep it as simple as possible
* ↳ the above chain of programs is a great example of how data is getting populated and passed and output is produced.

Q) How is output of 1 program written by someone else is taken as input for another program written by some other person?

P-T-O

Processes are communicating with each other super fast because they have a uniform interface.

And that interface is called a file (file descriptor) it is just a sequence of ordered bytes. And because this file (Basically a seqence of bytes) is such a simple interface, that it is used as an interface for.

→ communication channel to another process (UNIX Socket, stdin, stdout)

→ tcp connection etc.

We take this for granted, we should appreciate this more

www → internet

URL is the common interface

URL → { hyperLink1 → hyperLink2 → hyperLink3 →

So we keep passing URLs as input giving us the web page as the output

Pipes can be used to directly write stdout of one process into stdin of another process. And we don't even need to write the intermediate data into disk just writing to a small in-memory buffer is fine

Limits of stdin / stdout

- while am transferring stdout of 1 process into stdin of another, at that time I cannot take interactive inputs.
- If am expecting to merge input from network calls I cannot do that

Transparency & experimentation.

Advantages of Unix tools

*) Process 1 → | process 2 | process 3

↳ Input files → Input of 1 process into that of another process.

This input file is immutable and hence any process can use it any way it wants without the fear of altering it

* ~~Pipes~~ process 1 | process 2 | - - - | process i → less

We can do debugging at any step to check at which point it's failing

(Disadvantage) → (Unix runs of single machine)

(this is where Map Reduce comes into play)

↘ Finally makes the entry

(MapReduce & Distributed File Systems)

* MapReduce is a bit like Unix tools but distributed across multiple machines which unix was lacking.

In UNIX whatever is considered 1 process is one 1 map reduce job and its output is passed to another job.

(PTO)

In UNIX tools stdin and stdout as input and output, MapReduce jobs read and write files on distributed filesystem. In Hadoop implementation on Map-Reduce is called.

HDFS → Hadoop Distributed File System same as

GFS → Google File System.

(Redshift or Amazon S3) can also be used

HDFS → Storage structure is just like Cassandra or DynamoDB.

(Shared nothing Architecture)

NameNode → ~~Node~~ It's like Zookeeper which has info where which files reside on which machine

HDFS can have 10k machines storing ~~a~~ 100 PB of data

~~URL Systems~~ whi

(Map Reduce Job Execution.)

Lets take the example of the access-log ~~a~~ top K url examples.

Step1 → Take a batch of records

Step2 → map each record to (K,V) → (url, null) ⟹ MapStep

Step3 → Sort by (K,V)

Step4 → call reducer to group by (~~K Count~~) K → count → aggregation. ⟹ Reduce step

Step 2 & Step 4

⇓ ⇓

Mapper Reducer

↘ ↙

where you write your custom logic

Mapper → For every mapper input record, it creates a bunch of (K,V) pairs and might generate none.

Reducer → takes the (K,V) pair and aggregates [url → count] pair.

If you remember, we still have to do another sort by ~~distinct~~ count

Mapper → Prepare data ~~K~~ (url, count) → 2nd map Reduce job

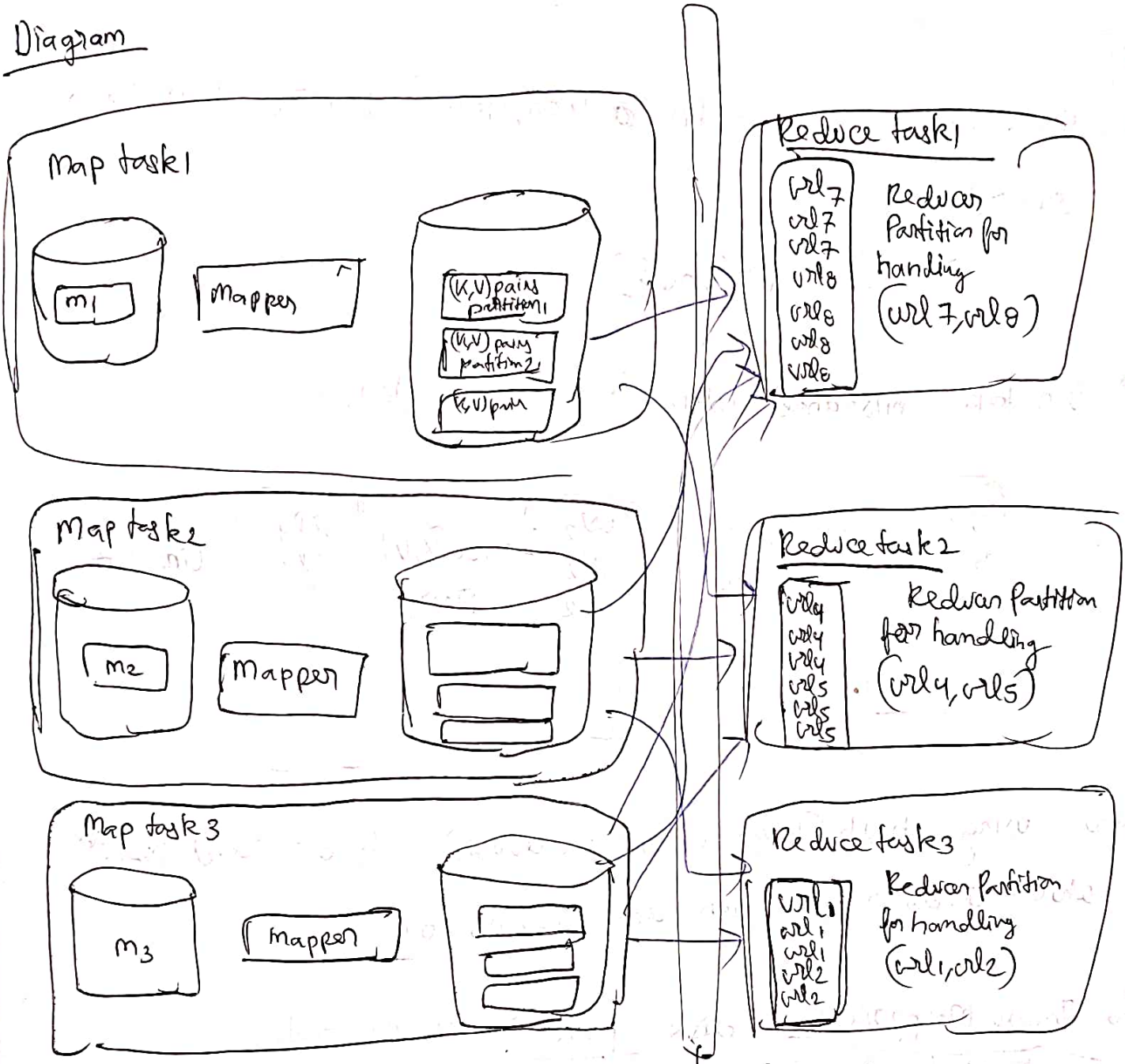
reducer → sorts the data by count

Distributed Execution of MapReduce.

- MapReduce wins over UNIX because it can parallelize tasks.
- In Hadoop MapReduce mapper and reducer are each Java class that implement a ~~callback~~ callable interface

* Major win is that in HDFS, Hadoop assigns mapper task to those machines which have a replica of the input file which is to be processed. [Increased Data Locality, Putting computation near data, No Network call]

Diagram



↳ this is basically reducer polling from corresponding data from the mapper's local disk

Steps of Execution

1) Let's say file is 3GB big. It's broken into blocks lets say 3 blocks 1GB each.

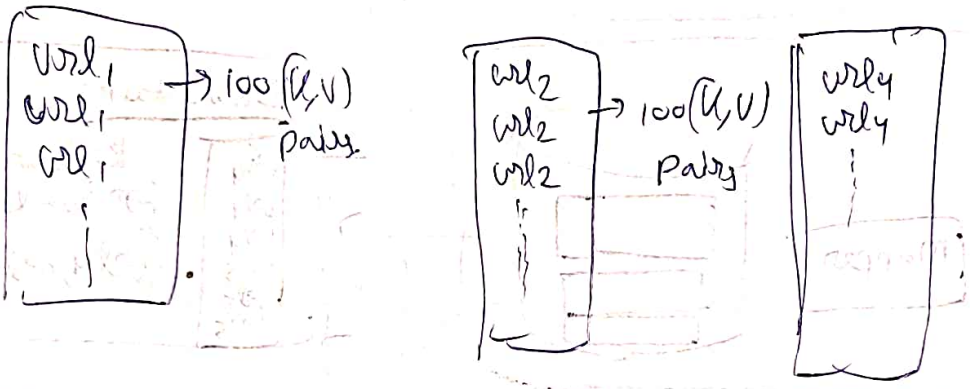
2) Each mapper instance who have block1, block2, block3 1GB each, is assigned a map task

3) Each map task downloads the JAR of the map function which has to be executed

4). Each Mapper generates a (key, value) pairs from each record.

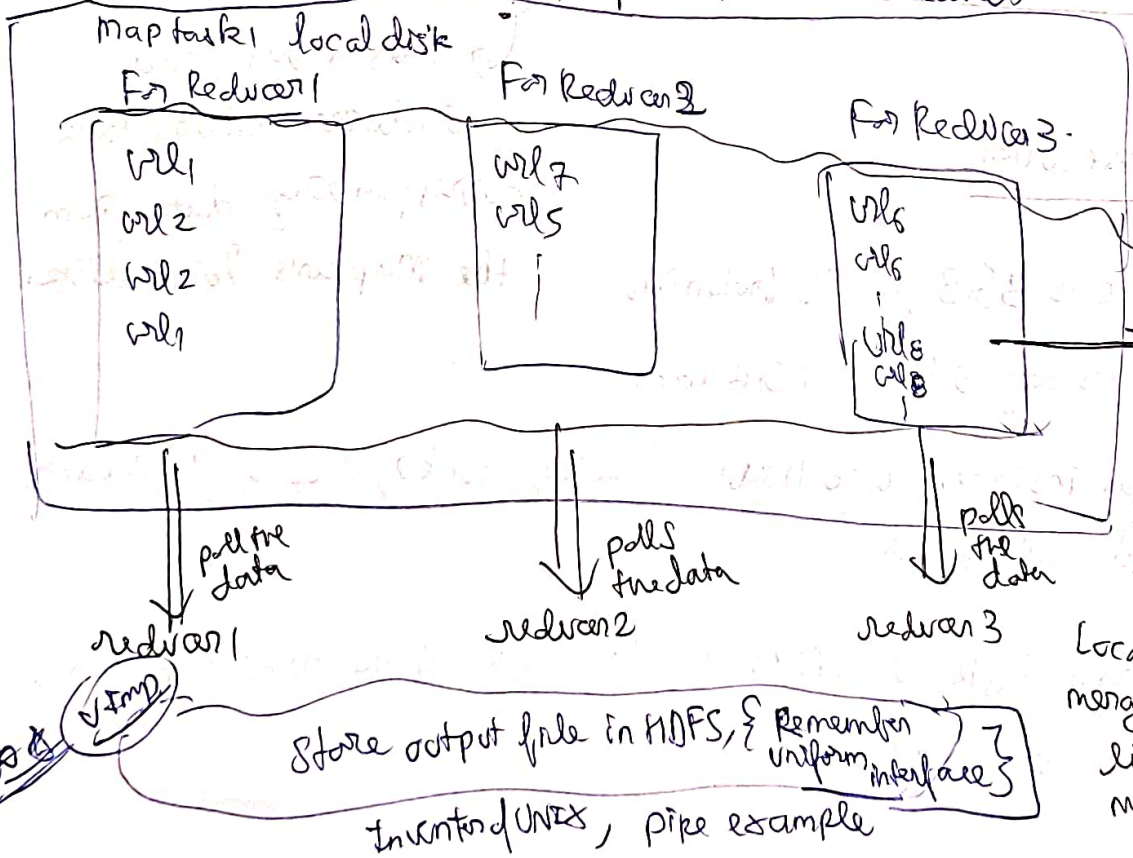
record → (url, null)

Lets take instance which has map task 1.



Now using hash(key) % (Num of Reducers) its decided which reducer partition data will go to.

So Inside Mapper's local disk partitions are created



In UNIX the "|" pipe operator to pass output of one process to input of another is using a small in-memory buffer.

In MapReduce we saw how output is stored smartly in HDFS only for any other Job flow to pick it up.

Airflow is similar just requires DAG's to process it